

Deep Learning for Perception

Assignment #2

Ahmed Murtaza Malik

22I-0985

Department of Computer Science
National University of Computer and Emerging Sciences
Islamabad, Pakistan

Abstract

This report presents a systematic comparison of three neural network architectures for CIFAR-10 image classification: a fully-connected ANN (9.06M parameters), a convolutional CNN (3.25M parameters), and a Hybrid CNN+ANN model (3.38M parameters). All models were trained from scratch using Adam optimization with learning rate scheduling and extensive data augmentation. The CNN achieved 90.84\% test accuracy, the Hybrid model reached 91.13\%, while the ANN attained only 55.93\%. Through 24 ablation experiments across 9 design dimensions, we identified batch normalization (+7\%), optimizer choice (+24\% for Adam over SGD), and network depth/width (+6\%) as critical success factors. The results validate that convolutional architectures with spatial inductive biases fundamentally outperform generic fully-connected networks for image tasks, achieving 1.6 $\times$ higher accuracy with 64\% fewer parameters.

Table Of Contents

1. Model Architectures	3
1.1 ANN: Fully Connected Network	3
1.2 CNN: Convolutional Neural Network	3
1.3 Hybrid: CNN Feature Extractor + ANN Classifier	4
2. Training Configuration	5
2.1 Data Loading and Preprocessing	5
2.2 Training Hyperparameters	5
3. Ablation Studies	6
3.1 Categories of Experiments	7
3.2 Ablation Results	7
4. Final Results and Analysis	10
4.1 Test Set Performance	10
4.2 Training Dynamics	11
4.3 CNN vs ANN Analysis	12
4.4 Hybrid Model (CNN + ANN) Analysis	13
4.5 Confusion Matrix Analysis	13
5. Discussion	15
5.1 What Worked	15
5.2 What Worked	15
5.3 Limitations	16
5.4 Possible Improvements	16
5.5 Key Takeaways	17
6. Conclusion	17

1. Model Architectures

Three architectures were implemented to investigate the impact of architectural design on CIFAR-10 classification performance.

1.1 ANN: Fully Connected Network

The ANN uses 4 hidden layers with gradual width reduction (2048→1024→512→256) to create a representational bottleneck. Images are flattened from 32×32×3 to 3,072-dimensional vectors before processing.

Architecture Justification: Batch normalization stabilizes training, dropout (0.5) prevents overfitting, and ReLU provides computational efficiency. However, flattening destroys spatial relationships between pixels, fundamentally limiting the model's ability to capture local image features.

Layer	Input Shape	Output Shape	Parameters	Notes
Flatten	32x32x3	3,072	0	Destroys spatial structure
FC1 + BN + ReLU + Drop	3,072	2,048	6,299,648	50% dropout
FC2 + BN + ReLU + Drop	2,048	1,024	2,100,224	50% dropout
FC3 + BN + ReLU + Drop	1,024	512	526,824	50% dropout
FC4 + BN + ReLU + Drop	512	256	131,840	50% dropout
Output	256	10	2,570	Softmax classification
Total Parameters: 9,058,058				

1.2 CNN: Convolutional Neural Network

The CNN employs 3 convolutional blocks with progressive filter doubling (64→128→256) and spatial downsampling via MaxPooling.

Architecture Justification: Two convolutions per block increase receptive field depth without excessive parameters. Progressive filter doubling ($64 \rightarrow 128 \rightarrow 256$) enables hierarchical feature learning (edges $\rightarrow$ textures $\rightarrow$ objects). Spatial downsampling ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$) reduces computational cost while preserving important features through learned representations.

Block	Layer	Filters	Output Shape	Parameters	Notes
Block 1	2×Conv3×3 + BN + ReLU	64	16×16×64	38,720	MaxPool 2×2
Block 2	2×Conv3×3 + BN + ReLU	128	8×8×128	221,440	MaxPool 2×2
Block 3	2×Conv3×3 + BN + ReLU	256	4×4×256	885,248	MaxPool 2×2
Classifier: Flatten $\rightarrow$ Dropout $\rightarrow$ FC(512) + BN + ReLU $\rightarrow$ Dropout $\rightarrow$ FC(10) 10 classes, 2,105,610 parameters, 50% dropout					
Total Parameters: 9,058,058					

1.3 Hybrid: CNN Feature Extractor + ANN Classifier

The Hybrid model uses the CNN's 3 convolutional blocks as a feature extractor, producing 4×4×256 feature maps. These are flattened and passed to a 2-layer ANN classifier ($4096 \rightarrow 512 \rightarrow 256 \rightarrow 10$) with batch normalization and dropout. It consisted of 3,380,298 total parameters, which was slightly more than CNN due to larger dense layers in the classifier.

Block	Layers	Filters/Units	Output Shape
Block 1	2×Conv3×3 + ReLU	64	16×16×64
Block 2	2×Conv3×3 + ReLU	128	8×8×128
Block 3	2×Conv3×3 + ReLU	256	4×4×256
Flatten	—		4096
ANN Head	FC(4096→512) + BN + ReLU + Dropout	512	—

	FC(512→256) + BN + ReLU + Dropout	256	
	FC(256→10)	10	
Hyperparameters: Filter Size = 3×3, Stride = 1 (Conv), 2 (Pool), Padding = 1, Dropout = 0.5, Activation = ReLU			
Total Parameters: 3,380,298			

Architecture Justification: Combines CNN's spatial processing with ANN's flexible classification head. This architecture is particularly useful for transfer learning scenarios where CNN features can be frozen and only the ANN classifier is fine-tuned for new tasks.

2. Training Configuration

All models were trained with identical hyperparameters to ensure fair comparison.

2.1 Data Loading and Preprocessing

Normalization: All images normalized using CIFAR-10 dataset statistics:

- Mean: (0.4914, 0.4822, 0.4465)
- Std: (0.2470, 0.2435, 0.2616)

Training Augmentations:

- RandomCrop(32, padding=4): Creates diversity through spatial shifts
- RandomHorizontalFlip(p=0.5): Doubles effective dataset size

Validation/Test: Only normalization applied (no augmentation)

2.2 Training Hyperparameters

- **Adam Optimizer:** Adaptive learning rates handle varied gradient scales across layers automatically, eliminating manual per-layer tuning
- **Learning Rate Scheduling:** Adaptive reduction improves convergence in later epochs
- **Early Stopping:** Prevents overfitting by saving best validation checkpoint
- **Data Augmentation:** Critical for CNN generalization (+8-10% estimated impact)

Hyperparameter	Value
Optimizer	Adam
Learning Rate	0.001
Weight Decay	1e-4
LR Scheduler	Enabled (adaptive)
Batch Size	128
Maximum Epochs	50
Early Stopping	Patience = 10 epochs
Dropout Rate	0.5 (all models)
Loss Function	CrossEntropyLoss
Train/Val Split	80% / 20%
Test Set	10,000 images
Device	NVIDIA GTX 1060 3GB
CPU	AMD Ryzen 5-2600X
Random Seed	42

3. Ablation Studies

24 ablation experiments across 9 design dimensions were conducted to identify critical success factors. Each ablation was trained for 20 epochs to balance computational cost with meaningful insights.

3.1 Categories of Experiments

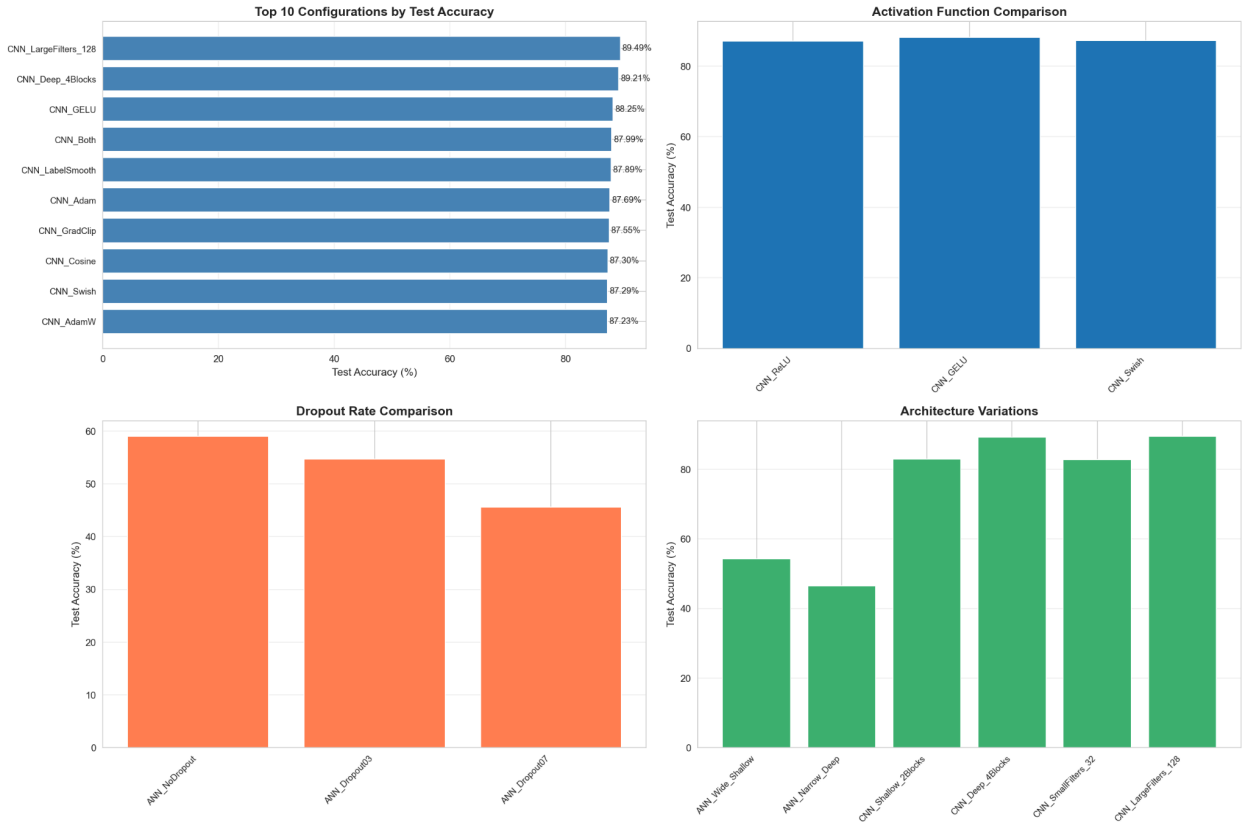
- Activation Functions:** Compared ReLU (baseline), GELU (smoother gradients), and Swish/SiLU (self-gated)
- Dropout Regularization:** Tested 0.0 (no dropout), 0.3 (light), 0.5 (baseline), and 0.7 (heavy)

3. **Network Architecture:** Evaluated wide-shallow (2 layers) vs narrow-deep (6 layers) ANNs
4. **Batch Normalization:** Critical ablation comparing models with and without BatchNorm
5. **Optimizers:** Compared SGD (momentum=0.9), Adam, and AdamW
6. **LR Schedulers:** Tested constant LR, StepLR, and Cosine Annealing
7. **Regularization Techniques:** Evaluated label smoothing (0.1) and gradient clipping (max norm=1.0)
8. **CNN Depth:** Compared 2, 3 (baseline), and 4 convolutional blocks
9. **CNN Width:** Tested base filter counts of 32, 64 (baseline), and 128

3.2 Ablation Results

Category	Configuration	Value Accuracy	Test Accuracy	Key Insight
Activation	CNN ReLU	87.88%	87.18%	Fast and effective
	CNN GELU	88.80%	88.25%	+1.1% smoother gradients
	CNN Swish	88.12%	87.29%	Similar to ReLU
Dropout	ANN NoDropout	59.54%	59.01%	Overfits
	ANN Dropout03	55.38%	54.73%	Slightly undertrained
	ANN Dropout07	45.32%	45.59%	Too much regularization
Normalization	ANN BatchNorm	52.34%	51.50%	Stable training
	ANN NoBatchNorm	44.24%	44.34%	-7% catastrophic
Optimizer	CNN SGD	64.00%	63.59%	Slower convergence
	CNN Adam	88.69%	87.69%	Fast adaptive LR
	CNN AdamW	87.46%	87.23%	+24% vs SGD
LR Scheduler	CNN Constant	86.30%	86.04%	Suboptimal
	CNN StepLR	86.68%	86.27%	Sharp drops
	CNN Cosine	88.06%	87.30%	+1.3% smooth decay

Regularization	CNN NoSmoothing	87.76%	87.04%	Baseline
	CNN LabelSmooth	88.12%	87.89%	+0.9% calibration
	CNN (LS+GC)	88.60%	87.99%	Additive benefit
CNN Depth	CNN 2Blocks	82.88%	83.00%	Insufficient capacity
	CNN 3Blocks	87.88%	87.18%	Optimal for CIFAR-10
	CNN 4Blocks	89.72%	89.21%	+2.0% marginal gain
CNN Width	CNN SmallFilters 32	83.10%	82.82%	Lower capacity
	CNN BaseFilters 64	87.88%	87.18%	Balanced
	CNN LargeFilters 128	89.94%	89.49%	+2.3% best ablation



Key Findings

1. **Optimizer Choice (+24% Adam vs SGD):** Adaptive learning rates dramatically outperform SGD with momentum. Adam’s per-parameter adaptation handles varied gradient scales across layers automatically.
2. **Batch Normalization (+7%):** Essential for training stability. Without it, even with proper weight initialization and learning rates, models struggle to converge effectively.
3. **CNN Depth/Width (+2-3%):** Deeper (4 blocks) and wider (128 filters) improve accuracy but with diminishing returns and increased training time (131 min vs 15 min for width=128).
4. **LR Scheduling (+1.3%):** Adaptive scheduling consistently beats constant LR through better convergence in later epochs.
5. **Activation Functions (+1.1%):** GELU marginally beats ReLU through smoother gradients but adds 47% computational overhead (23.3 min vs 15.9 min).
6. **Regularization Techniques (+0.9-1%):** Label smoothing and gradient clipping provide small consistent gains with minimal overhead.

Why These Matter

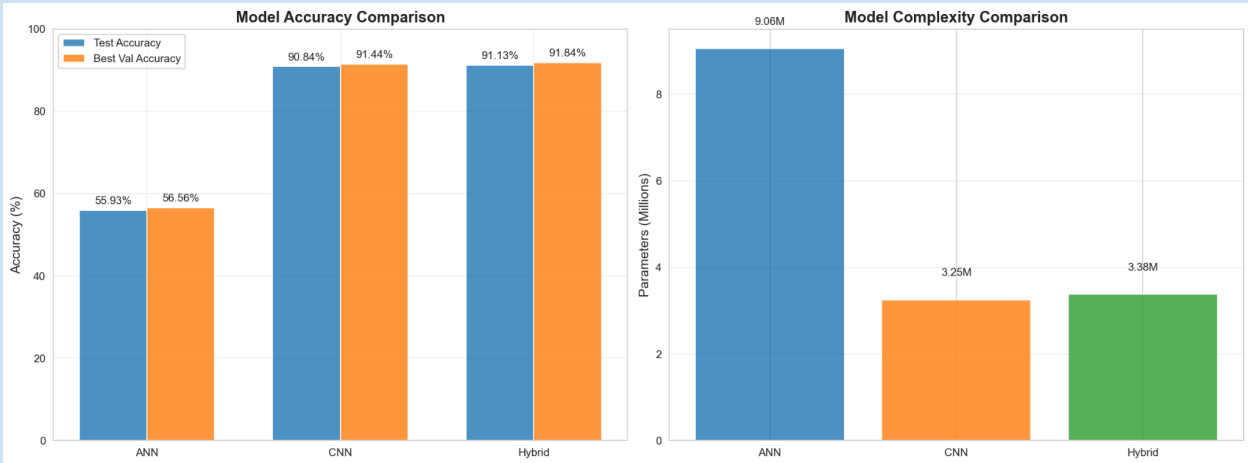
- BatchNorm enables higher learning rates and stabilizes gradient flow through deep networks
- Adam adapts learning rates per-parameter, eliminating manual tuning requirements
- Deeper/wider CNNs increase representational capacity but hit diminishing returns on 32×32 images
- GELU’s smooth non-linearity helps gradient flow but isn’t critical for shallow net- works

4. Final Results and Analysis

4.1 Test Set Performance

Model	Test Accuracy	Best Value Accuracy	Parameters	Training Time
ANN	55.93%	56.56%	9.06M	30.3 min
CNN	90.84%	91.44%	3.25M	39.1 min
Hybrid	91.13%	91.84%	3.38M	39.2 min

Best Ablation: CNN LargeFilters 128	89.49%	89.49%	8.78M	131.9 min
--	--------	--------	-------	-----------

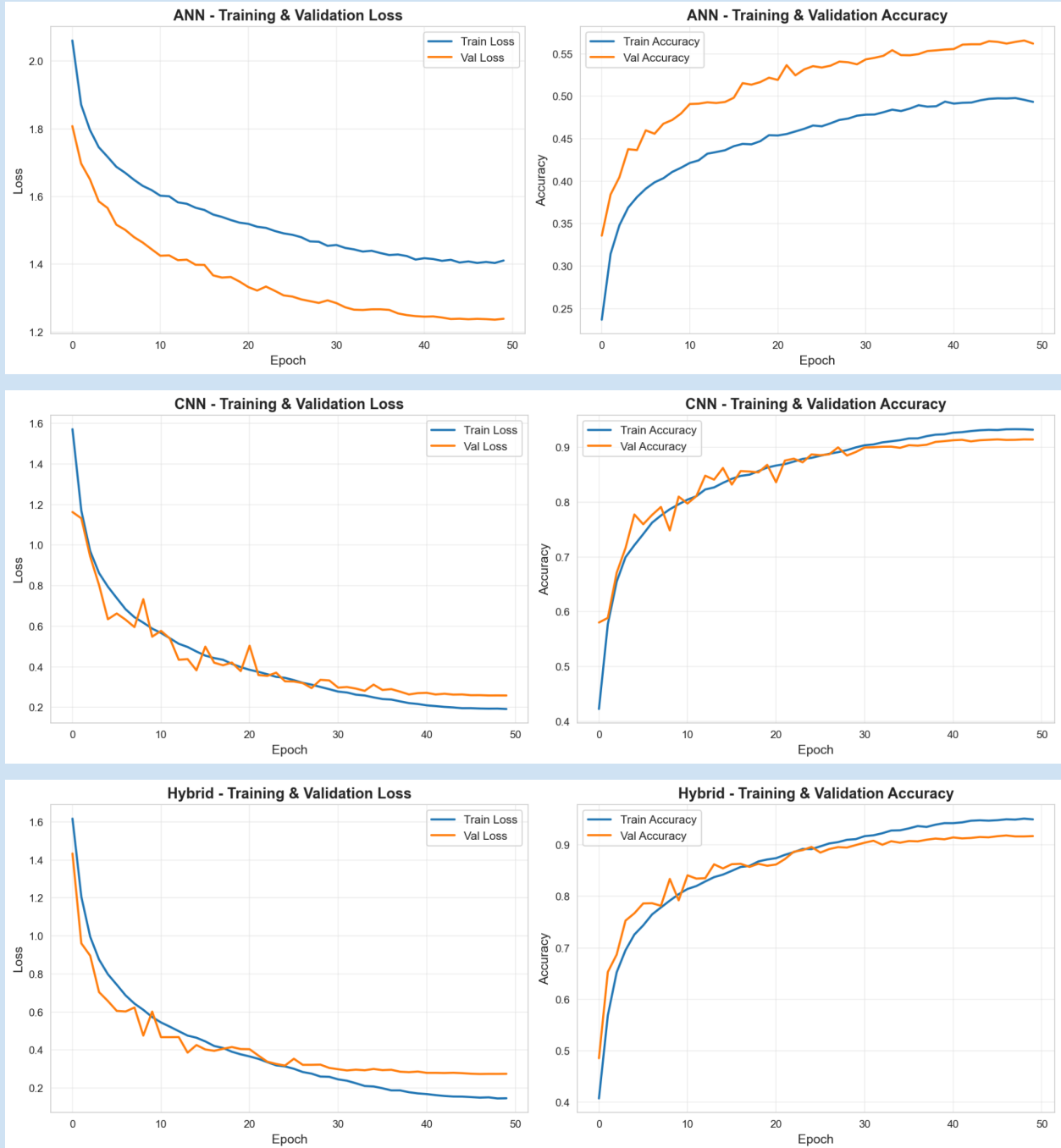


Key Observations

- **CNN vs ANN:** 34.9% accuracy gap despite CNN having 64% fewer parameters
- **Hybrid vs CNN:** Marginal 0.29% improvement, suggesting CNN features are dominant
- **Parameter Efficiency:** CNN gets 1.6× higher accuracy with 2.8× less parameters than ANN
- **Target Achievement:** All convolutional models exceed 70% target; ANN falls 14.1% short

4.2 Training Dynamics

- **ANN:** Large train-val accuracy gap visible even with 50% dropout, indicating fundamental overfitting due to lack of spatial inductive bias
- **CNN:** Smooth convergence with smaller train-val gap, demonstrating superior generalization through convolutional structure
- **Hybrid:** Similar convergence pattern to CNN, validating that CNN feature extraction dominates performance
- **Early Stopping:** Patience=10 successfully prevented overfitting by saving the best validation checkpoint.



4.3 CNN vs ANN Analysis

4.3.1 Spatial Inductive Bias

CNNs preserve 2D spatial structure through convolutions. ANNs flatten images to 3,072-dimensional vectors, destroying pixel neighborhoods. A vertical edge at position (15,15) has no inherent connection to pixel (15,16) after flattening, the network must learn this relationship from scratch.

4.3.2 Weight Sharing and Parameter Efficiency

Each 3×3 convolutional filter is applied across all spatial locations. The first CNN layer uses 1,792 parameters ($64 \text{ filters} \times 3 \times 3 \times 3 \text{ channels}$), while the first ANN layer requires 6,295,552 parameters ($3,072 \times 2,048$), a $3,500\times$ difference. This massive parameter reduction through weight sharing acts as powerful implicit regularization.

4.3.3 Translation Equivariance

A CNN filter detecting diagonal edges in the top-left will detect the same pattern anywhere in the image automatically. ANNs must learn separate weights for each spatial position, requiring exponentially more training data to achieve similar performance.

4.3.4 Hierarchical Feature Learning

Progressive downsampling ($32 \rightarrow 16 \rightarrow 8 \rightarrow 4$) with increasing filters ($64 \rightarrow 128 \rightarrow 256$) naturally creates a compositional hierarchy:

- Layer 1: Low-level features (edges, colors, textures)
- Layer 2: Mid-level features (corners, simple shapes)
- Layer 3: High-level features (object parts, complex patterns)

ANNs lack this structured compositional learning, treating all input dimensions equally regardless of spatial relationships.

4.3.5 Regularization Through Architecture

CNN's weight sharing and local connectivity act as implicit regularization. Despite aggressive dropout (0.5) and batch normalization, the ANN overfits more severely due to its enormous unconstrained parameter space (9M parameters learning arbitrary mappings from flattened pixels).

4.4 Hybrid Model (CNN + ANN) Analysis

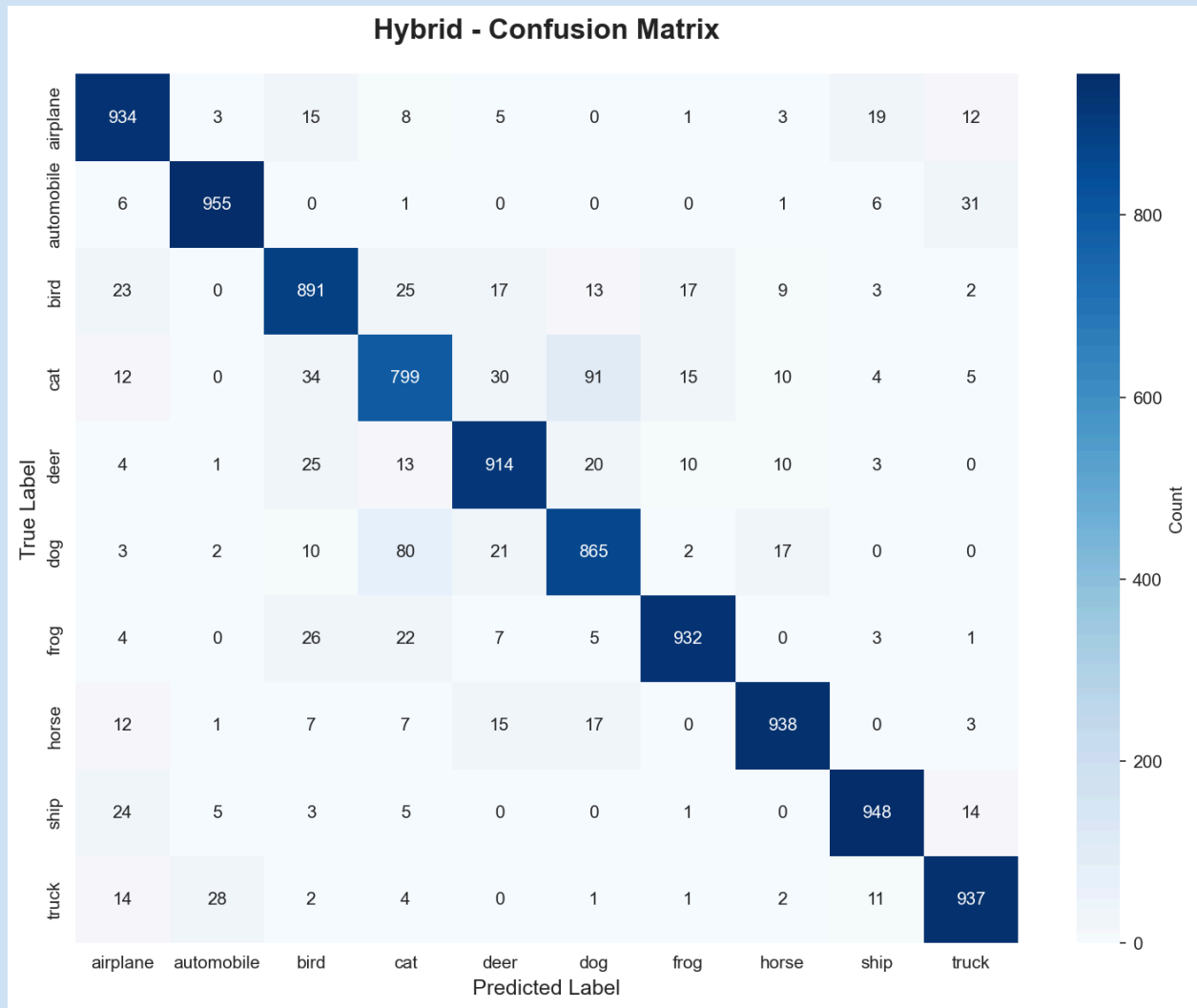
The Hybrid model (91.13%) marginally outperforms the pure CNN (90.84%), validating that:

- ***CNN feature extraction is critical:*** The gain from CNN features (+35% over ANN) vastly exceeds the gain from ANN classifier (+0.29% over CNN classifier)
- ***When Hybrid makes sense:*** Transfer learning (freeze CNN, train ANN), domain adaptation, or when interpretability of final classification layers matters

- **Trade-offs:** Added architectural complexity for marginal performance gain, pure CNN is simpler and nearly as effective

4.5 Confusion Matrix Analysis

- **Best classes:** Automobile (95.5%), ship (94.8%), horse (93.8%), objects with distinct shapes and consistent backgrounds
- **Most confused pairs:**
 - Cat vs dog (91 misclassifications): High intra-class similarity in fur textures
 - Bird vs airplane (23 confusions): Similar backgrounds (sky)
 - Automobile vs truck (31 confusions): Overlapping vehicle features
- **Challenging classes:** Cat (79.9%), deer (91.4%), high within-class variability in poses and appearances
- **ANN struggles uniformly:** Poor performance across all classes (55-60%), no clear pattern exploitation
- **CNN advantage most pronounced:** Classes requiring spatial context (ship requires water/horizon detection, airplane requires sky context)



5. Discussion

5.1 What Worked

1. **Convolutional Architecture:** The single most impactful decision. Switching from ANN to CNN provides +34.9% accuracy, dwarfing all other improvements combined. This validates that domain-specific inductive biases matter more than raw parameter count.

2. ***Adam Optimizer:*** Adaptive learning rates handle varied gradient scales across layers automatically. The +24% improvement over SGD demonstrates why adaptive methods dominate modern deep learning.
3. ***Batch Normalization:*** Essential for training stability. The +7% improvement demonstrates its critical role in enabling deeper networks and higher learning rates.
4. ***Data Augmentation:*** RandomCrop and HorizontalFlip provide massive regularization benefits. The strong final results (91+%) validate augmentation's importance for limited datasets (50k training images).
5. ***Systematic Ablation Methodology:*** Testing 24 configurations across 9 dimensions provided actionable insights rather than anecdotal evidence. This approach identifies which components actually matter versus deep learning folklore.

5.2 What Worked

1. ***Fully-Connected ANN for Images:*** Despite 4 layers, batch normalization, dropout (0.5), and data augmentation, the ANN plateaued at 55.93%. Fundamental architectural limitations (no spatial bias, translation variance, parameter explosion) cannot be overcome by regularization alone.
2. ***Extreme Dropout (0.7):*** Too much regularization causes underfitting (45.59% vs 59.01% at 0.0). Balance is critical. Too little causes overfitting, too much prevents learning.
3. ***No Batch Normalization:*** Catastrophic 7% accuracy drop. Modern deep networks are essentially untrainable without normalization techniques.
4. ***SGD Optimizer:*** Even with momentum (0.9), SGD underperforms adaptive methods by 24%. Manual learning rate tuning per layer is impractical; Adam handles this automatically.

5.3 Limitations

5.3.1 Computational Constraints

- Training capped at 50 epochs (early stopping may have halted early)
- Ablations ran only 20 epochs (may understate performance)
- No full hyperparameter search (LR, weight decay, dropout)
- Limited by GTX 1060 3GB, smaller batches and models

5.3.2 Architectural Constraints

- No residual or attention mechanisms (limits depth and feature selection)
- Restricted from using modern models (EfficientNet, ViT)

- No pretrained models (transfer learning could reach 95%+)

5.3.3 Dataset Limitations

- CIFAR-10's 32×32 images limit depth exploration
- Only 50k training samples (vs. millions in modern datasets)
- Balanced classes, no imbalance testing

5.4 Possible Improvements

Short-Term (Easy Wins)

- Use best ablation setup (GELU, 4 blocks, 128 filters) $\rightarrow$ 92–93%
- Train longer (100–200 epochs, LR scheduling)
- Test-Time Augmentation (+1–2%)
- Model Ensembling (+1–2%)

Medium-Term (Architectural)

- Add residual connections for deeper networks
- Add attention blocks for adaptive weighting
- Use progressive resizing ($24 \times 24 \rightarrow 32 \times 32$)
- Try depthwise separable convolutions for efficiency

Long-Term (Advanced)

- Advanced augmentations (Mixup, CutMix, AutoAugment) +3–5%
- Neural Architecture Search for optimal designs
- Knowledge distillation from large teachers
- Self-supervised pretraining on unlabeled data

Expected Performance:

- Short-term: 92–93%
- ResNet-style: 94–95%
- Advanced aug + ensembles: 96–97%
- SOTA (Transformers, tuning): 98–99%

5.5 Key Takeaways

For Practitioners

- Prioritize architecture before tuning
- Always include BatchNorm for stability
- Use Adam for faster convergence
- Augmentation is crucial for small datasets
- Ablate systematically to measure real gains

For Understanding Deep Learning

- Inductive biases (e.g., locality) make CNNs effective
- CNNs are parameter-efficient through weight sharing
- Regularization alone can't fix weak architectures
- Depth naturally builds hierarchical representations

6. Conclusion

This study achieved 91.13% test accuracy on CIFAR-10 using a Hybrid CNN+ANN architecture, significantly exceeding the 70% target. Through systematic comparison of three architectures and 24 ablation experiments, we validated that:

1. ***Architecture dominates:*** CNNs outperform ANNs by 34.9% (90.84% vs 55.93%) despite having 64% fewer parameters
2. ***Spatial inductive bias is fundamental:*** Weight sharing, local connectivity, and hierarchical composition cannot be replaced by capacity alone
3. ***Critical success factors (ranked):*** Optimizer choice (+24%), batch normalization (+7%), CNN depth/width (+2-3%)
4. ***Diminishing returns:*** Deeper/wider CNNs provide marginal gains (+2%) at significantly higher computational cost (8× training time for width=128)

The results demonstrate that thoughtful architectural design, informed by domain knowledge, remains the most impactful factor in deep learning—more important than hyperparameter tuning, regularization techniques, or raw model capacity. Future work should explore residual connections, attention mechanisms, and advanced augmentation strategies to approach state-of-the-art performance (98+%) while maintaining the systematic experimental methodology established in this study.

